

Reviewer Pack — Symmetric Group Fourier Coefficient Gap in Disjointness Comm...

Entry 7003ce43815a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 17:22:51 UTC

Symmetric Group Fourier Coefficient Gap in Disjointness Communication Matrices

Entry ID: 7003ce43815a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 17:22:51 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For any $n \times n$ disjointness matrix M , the maximum Fourier coefficient $|\chi_\lambda(M)|$ over irreducible symmetric group characters χ_λ satisfies $|\chi_\lambda(M)| \geq n^2/2$ for $\lambda = (n-1,1)$ and $|\chi_\lambda(M)| \leq n/2$ for $\lambda = (n)$.

Rationale (proposer's reasoning):

Symmetric group representations capture combinatorial symmetries of disjointness matrices. The $(n-1,1)$ partition corresponds to transpositions, which encode pairwise conflicts central to disjointness. The (n) partition corresponds to the trivial representation, which reflects global consistency. The conjecture links representation-theoretic structure to communication complexity via Fourier analysis.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cc3f7a3df88ea6a5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False
    return True

def generate_primes(n=30):
    primes = []
    num = 2
    while len(primes) < n:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def random_disjointness_matrix(n, seed=None):
    if seed is not None:
        random.seed(seed)
    M = [[0] * n for _ in range(n)]
    pairs = list(combinations(range(n), 2))
    random.shuffle(pairs)
    for i, (a, b) in enumerate(pairs[:n]):
        M[a][b] = M[b][a] = i + 1
    return M

def character_table(n):
    table = [[0] * n for _ in range(n)]
    table[0][0] = 1
    if n == 1:
        return table
    for k in range(1, n):
        for l in range(k + 1):
```

```

        sum_val = 0
        for j in range(l + 1):
            binom = math.comb(l, j)
            sign = (-1) ** (l - j)
            if k == l:
                char_table_n_minus_k_l = 1 if j % 2 == 0 else 0
            else:
                char_table_n_minus_k_l = character_table(n - k)[j][j]
            sum_val += sign * binom * char_table_n_minus_k_l
        table[k][l] = sum_val / math.sqrt(k + 1)
    return table

def fourier_coefficient(M, lambda_):
    n = len(M)
    char_table = character_table(n)
    chi_lambda = 0
    for i in range(n):
        for j in range(n):
            chi_lambda += M[i][j] * char_table[lambda_][i][lambda_[j]]
    return abs(chi_lambda / (n ** 2))

def run_trial(seed: int) -> dict:
    n = random.choice([5, 10, 15, 20, 30, 40])
    M = random_disjointness_matrix(n, seed)
    lambda_l = [n - 1] + [1]
    lambda_n = [n]
    chi_lambda_l = fourier_coefficient(M, lambda_l)
    chi_lambda_n = fourier_coefficient(M, lambda_n)
    metric_value = max(chi_lambda_l, chi_lambda_n)
    conjecture_holds = chi_lambda_l >= n**2 / 2 and chi_lambda_n <= n / 2
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "Fourier Coefficient Gap",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or generate_primes()
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric = sum(r["metric_value"] for r in results) / len(results)
    std_metric = math.sqrt(sum((r["metric_value"] - mean_metric) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    else:
        print(f"RESULT: NOT SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction} counterexample={counterexample}")

```

```

first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c28d86c4.py", line 94, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c28d86c4.py", line 77, in run_trial
    chi_lambda_1 = fourier_coefficient(M, lambda_1)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c28d86c4.py", line 65, in fourier_coefficient
    char_table = character_table(n)
                  ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c28d86c4.py", line 58, in character_table
    char_table_n_minus_k_l = character_table(n - k)[j][j]
                              ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c28d86c4.py", line 58, in character_table
    char_table_n_minus_k_l = character_table(n - k)[j][j]
                              ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c28d86c4.py", line 58, in character_table
    char_table_n_minus_k_l = character_table(n - k)[j][j]
                              ^^^^^^^^^^^^^^^^^^^^^
[Previous line repeated 10 more times]
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before completing; unable to verify conjecture's validity | next: Fix the character table generation to handle index ranges properly and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	40096
2	preregistration	ollama_remote	qwen3:8b	0	0	25334

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	qwen3:8b	0	0	20695
4	novelty	ollama_remote	qwen3:8b	0	0	14059
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	46009
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12407
7	judge	ollama_remote	qwen3:8b	0	0	17767

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 176367 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/7003ce43815a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7003ce43815a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7003ce43815a.tar.gz` (if generated)